

Reviewer Pack — Minimal Rank of Hodge Structures over Read-Twice Branching P...

Entry f4b220239417 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 13:42:22 UTC

Minimal Rank of Hodge Structures over Read-Twice Branching Programs

Entry ID: f4b220239417 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 13:42:22 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Read-Twice Branching Programs

Statement:

For any instance of a read-twice branching program with n gates, the minimal rank of its associated Hodge structure is equal to the width of the branching program, i.e., $E[\text{MinimalRank}(H)] = \Theta(n)$.

Rationale (proposer's reasoning):

The bridge between Hodge theory and complexity theory may reveal hidden structures in the hardness of computational problems. Hodge structures provide a rich algebraic framework for studying geometric objects, while read-twice branching programs are a fundamental model for bounded-error quantum algorithms. This conjecture aims to establish a quantitative relationship that could potentially lead to new insights into both fields.

Taxonomy category: COMM_DISJ (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 28c845b22e9a02f7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of seeds (≥ 24 out of 30) have a Hodge structure minimal rank equal to the width of the read-twice branching program with an absolute difference of ≤ 3 between the two, and no seed has a Hodge structure minimal rank greater than 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge structure" AND "read-twice branching programs" - "minimal rank" AND "algebraic geometry" AND "complexity theory" - "Hodge Theory" AND "branching program width"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    p = len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    augmented_matrix = [row + [b[i]] for i, row in enumerate(A)]
    for j in range(n):
        max_row = j
        for i in range(j+1, m):
            if abs(augmented_matrix[i][j]) > abs(augmented_matrix[max_row][j]):
                max_row = i
```

```

augmented_matrix[j], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[j]
pivot = augmented_matrix[j][j]
for i in range(j, n+1):
    augmented_matrix[j][i] /= pivot
for i in range(m):
    if i != j:
        factor = augmented_matrix[i][j]
        for k in range(j, n+1):
            augmented_matrix[i][k] -= factor * augmented_matrix[j][k]
return [row[-1] for row in augmented_matrix]

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    width = random.randint(2, n)

    # Generate a read-twice branching program
    branching_program = [[random.choice([0, 1]) for _ in range(width)] for _ in range(n)]

    # Compute the transition function
    transition_function = {}
    for i in range(n):
        for j in range(width):
            if branching_program[i][j] == 0:
                next_state = (i + 1) % n
            else:
                next_state = (n - i - 2) % n
            if (i, j) not in transition_function:
                transition_function[(i, j)] = []
            transition_function[(i, j)].append(next_state)

    # Compute the Hodge structure associated with the transition function
    hodge_structure = []
    for state in range(n):
        for i in range(width):
            if (state, i) not in transition_function:
                continue
            next_states = transition_function[(state, i)]
            row = [0] * n
            for next_state in next_states:
                row[next_state] += 1
            hodge_structure.append(row)

    # Compute the minimal rank of the Hodge structure
    hodge_rank = len(gaussian_elimination(hodge_structure, [0] * n))

    # Check if the conjecture holds
    conjecture_holds = abs(hodge_rank - width) <= 3 and hodge_rank <= 10

    return {
        "metric_name": "minimal_rank",
        "metric_value": hodge_rank,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": f"rank={hodge_rank}, expected={width}" if not conjecture_holds else ""
    }

```

```

    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, 'metric_name': '{result['metric_name']}', 'metric_value': {result['metric_value']}}}")
        results.append(result)

    mean = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(r["counterexample"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if r["conjecture_holds"] is False)
        print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44b82df6.py", line 108, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44b82df6.py", line 89, in run_trial
    hodge_rank = len(gaussian_elimination(hodge_structure, [0] * n))
                    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44b82df6.py", line 38, in gaussian_elimination
    augmented_matrix = [row + [b[i]] for i, row in enumerate(A)]
                        ~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture’s support conditions. | next: Investigate and fix the crash in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12392
2	propose	ollama_remote	glm4:latest	0	0	10332
3	propose	ollama_remote	glm4:latest	0	0	14071
4	preregistration	ollama_remote	glm4:latest	0	0	5867
5	novelty	ollama_remote	glm4:latest	0	0	4627
6	novelty	ollama_remote	glm4:latest	0	0	10498
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13758
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9025
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7149
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13224
11	judge	ollama_remote	glm4:latest	0	0	11602

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 112546 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/f4b220239417.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f4b220239417.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f4b220239417.tar.gz (if generated)